

Examen Java 2
Durée 2H

Exercice 1 : (2.5 points)

Le mécanisme des exceptions en Java permet de gérer de les erreurs pouvant survenir à l'exécution?	1./ Vrai 2./ Faux
Lorsqu'une séquence d'instructions est susceptible d'occasionner une erreur, on la place dans un bloc try .	1./ Vrai 2./ Faux
Une exception est une	1./ Classe 2./ Interface 3./ Classe abstraite 4./ Autre
Il se peut qu'une même instruction (ou suite d'instruction) soit susceptible de générer des exceptions différentes, nous pouvons dans ce cas placer plusieurs catchs la suite du même try .	1./ Vrai 2./ Faux
L'ordre des blocs catch est important.	1./ Vrai 2./ Faux

Exercice 2 : (2.5 points)

La classe de base des composants swing est JComponent.	1./ Vrai 2./ Faux
Quel est le nom de la classe Swing utilisée pour créer un frame?	1./ Window 2./ Frame 3./ JFrame 4./ SwingFrame
Chaque conteneur utilise un gestionnaire de placement (Layout) pour la disposition des composants qu'il contient.	1./ Vrai 2./ Faux
Quel objet peut être construit pour afficher un certain nombre de choix dans la fenêtre?	1./ JLabel 2./ JChoice 3./ JList 4./ JCheckbox
Pour organiser les boutons d'une calculatrice dans un conteneur le mieux c'est d'utiliser ?	1./ BorderLayout 2./ GridLayout 3./ CardLayout 4./ BorderLayout

Exercice 3 : (7 points)

Ecrire une classe java **Menu** contenant une méthode **saisirChoix** qui prendra en paramètre un entier **n** et renverra une valeur comprise entre 1 et **n**, tapée au clavier par l'utilisateur.

Les différentes erreurs qui pourront se produire seront :

- n est inférieur ou égal à 1
- l'utilisateur a entré un nombre qui n'est pas compris entre 1 et n
- l'utilisateur n'a pas entré un nombre

Chaque erreur devra être détectée par le programme et être signalée par une exception spécifique.

1./ Ecrire les classes différentes d'exception.

2./ Ecrire la classe **Menu**.



Examen Java 2 - Session
Durée 2H

Exercice 1 : (4 points)

Q1 :	Q2 :
On considère le code: <pre>class A { int i=5; void f(){ System.out.println(i); } } class B extends A { int i=4; void f(){ super.f(); } }</pre> Le code suivant: <pre>A a= new B(); a.f();</pre> (a) affiche 5 (b) affiche 4 (c) provoque une erreur à la compilation ou à l'exécution	<pre>class A{ String s="A"; void f(String s1){ System.out.println(s+" "+s1); } } class B extends A{ String s="B"; void f(String s1){ super.f(s+" "+s1); } } class C extends B{ String s="C"; void f(String s1){ super.f(s+" "+s1); } }</pre> le code <pre>A a=new C(); a.f("X");</pre> (a) Provoque une erreur à la compilation (b) Affiche C C C X (c) Affiche A B C X (d) Affiche A X

Exercice 2 : (4 points)

Ecrivez un programme Java qui permet de saisir, calculer le nombre des éléments nuls (=0) et afficher une matrice carrée n x n.

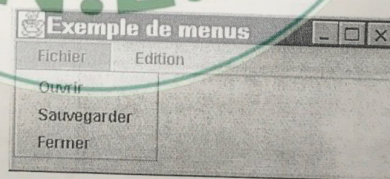
Exercice 3 : (4 points)

Dessiner les composants graphiques suivants (si possible) et identifier la classe java SWING correspondante (TextField, TextArea, List, Label, Button, Choice, Scrollbar, Checkbox);

Exercice 4 : (8 points)

Créer une fenêtre dérivée de la classe JFrame munie d'une barre de menus constituée :

- 1- d'un menu Fichier comportant les entrées : Ouvrir, sauvegarder et Fermer.
- 2- d'un menu Edition comportant les entrées : Copier et Coller.



On ne cherchera pas ici à traiter les actions correspondantes.

Question 1 (8 points) : des erreurs de compilation apparaissent dans le code ci-dessous. Proposez une correction du code sans supprimer des instructions.

```
public abstract class A {
    public abstract void m();
    public int v(int a){ return v*2;}
}
public class B extends A {
    int b;
    B () {
        this(1, 3);
    }
    B (int c) {
        This.b =b;
    }
}
```

```
public interface C {
    private int met(int a){}
}
public class D extends B, implements A {
    int c;
    public D(){ super("b"); }
}
public class Test{
    public static void main(String [ ] args){
        A a =new A(); C c = new B();
    }
}
```

Question 2 (12 pts) : On souhaite développer une petite application de conversion entre unités de mesures : Taille, poids, volume. On propose l'interface graphique suivant (Figure 1)

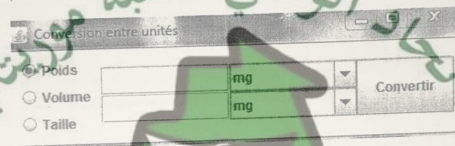


Figure 1

1. En utilisant swing, écrire un classe qui donne cette interface en respectant le même positionnement des composants ?
2. Ecrire le code nécessaire pour désactiver tous les autres boutons radio en activant un nouveau bouton radio (Un seul bouton radio doit être actif).
3. Ecrire le code nécessaire pour remplir les deux listes déroulantes (comboBox) avec les unités correspondant selon le choix sélectionné (poids (mg, cg, g), taille (mm, cm, m), volume (ml, cl, l)).
4. Ecrire le code nécessaire pour effectuer la conversion en cliquant sur le bouton convertir (voir les figures ci-dessous). On suppose que la valeur à convertir est celle dans le champ de saisie au dessus et son unité est donnée par la sélection dans la liste déroulante à sa droite. Le résultat est à afficher dans le champ de saisie en dessous selon l'unité choisie dans la deuxième liste.

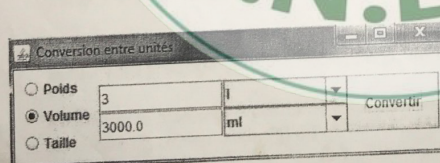


Figure 2

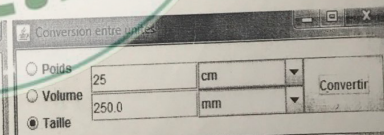


Figure 3



Examen Java2

Durée 2H

Mise en scène

J1624

Exercice 1 : (5 points)

Le mécanisme des exceptions en Java permet de gérer de les erreurs pouvant survenir à l'exécution?	1./ Vrai ✓ 2./ Faux
Lorsqu'une séquence d'instructions est susceptible d'occasionner une erreur, on la place dans un bloc try .	1./ Vrai ✓ 2./ Faux
Une exception est tout simplement une classe héritant de la classe Exception .	1./ Vrai ✓ 2./ Faux ✗
Il se peut qu'une même instruction (ou suite d'instruction) soit susceptible de générer des exceptions différentes, nous pouvons dans ce cas placer plusieurs catch à la suite du même try .	1./ Vrai ✓ 2./ Faux ✗
L'ordre des blocs catch n'est pas important.	1./ Vrai ✓ 2./ Faux ✗

Exercice 2 : (3 points)

Un Palindrome est une chaîne de caractère qu'on peut lire de gauche comme de droite, exemple: laval.
Ecrire une méthode **public boolean estUnPalindrome(String chaîne)** qui vérifie si chaîne est un palindrome.

Exercice 3 : (Calcul matriciel - 12 points)

Ecrire une classe appelée **calculMatriciel** contenant les méthodes suivantes :

A./ Afficher une matrice ligne par ligne
public void afficheMatrice(int[][] matrice)

B./ Saisir une matrice
public int[][] saisirMatrice(int[][] a, int[][] b)

C./ Somme de deux matrices a et b et retourne le résultat dans une matrice C
public int[][] somme(int[][] a, int[][] b)

D./ Echanger les valeurs de deux éléments **m[i1][j1]** et **m[i2][j2]**
public void echange (int[][] m, int i1, int j1, int i2, int j2)

E./ Utiliser la méthode précédente pour calculer et retourner la matrice transposée
public int[][] transpose(int[][] m)

F./ Produit de deux matrices (Si possible)
public int[][] produit(int[][] a, int[][] b)



Exercice 1 : (3 points)

Quelle classe n'a pas de classe mère ?	(a) Orpheline (b) String (c) Object ✓ (d) une classe abstraite ✗
Qu'est-ce qui est faux pour les interfaces ?	(a) Une I. peut être le type d'une référence ✗ (b) Une I. déclare des méthodes sans les implémenter ✓ (c) Une I. peut être implémentée (d) Une I. peut être instanciée
Pour la classe D définie comme suit : class D { public int x; public D() {x=3;}; public D(int a){this(); x=x+a;}; public D(int a, int b){this(b); x=x+a;}; qu'affichera le code suivant ? D a=new D(5); System.out.println(a.x);	(a) 1 (b) 2 (c) 3 ✓ (d) 4

Exercice 2 : (3 points)

Ecrivez un programme Java qui permet de saisir, transposer et afficher une matrice carrée $n \times n$.

Exercice 3 : (2 points)

Dessiner les composants graphiques suivants (si possible) et identifier la classe java SWING correspondante (TextField, TextArea, Liste, Label, Button, Choice, Scrollbar, Checkbox);

Exercice 4 : Réalisation d'une calculatrice (12 points)

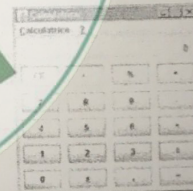
Cet exercice consiste à réaliser une calculatrice comme le montre la figure en face.

- Cette calculatrice doit permettre d'effectuer uniquement les 4 opérations arithmétiques (x, /, +, -) sur des nombres réels.
- Utiliser le mécanisme d'exception pour éviter la division par zéro.
- Le bouton CE sera utilisé pour effacer tout le contenu de la zone texte et elle ne doit être active que lorsqu'il y a du texte dans la zone du texte.
- Le bouton «flèche» sera utilisé pour effacer le dernier caractère ajouté dans la zone texte
- Il faut contrôler la saisie, l'idée : il faut seulement autoriser la saisie de valeurs dans la zone de texte via les touches de la calculatrice;
- On ne peut pas utiliser le bouton virgule s'il a déjà été utilisé pour la saisie en cours

Travail demandé:

1. Ecrire le code permettant de créer la fenêtre (préciser le nom de l'application et la taille)
2. Créer la barre de menu et y ajouter les deux menus visibles sur l'image (ajouter quitter à la fin du menu calculatrice)
3. Ajouter la barre de menu à la fenêtre précédemment créée;
4. Quelle disposition proposez vous pour organiser le restes de vos éléments graphiques sur la fenêtre? expliquer en donnant un schéma;
5. Dessiner les composants graphiques correspondant au schéma précédent;

Travail demandé: Dessiner les composants graphiques correspondant au schéma précédent; en tenant compte des règles précisées au début de l'exercice.



ISCAE

CC JAVA II

RT2 /IG2/ID2

211-1-RT 60

```
public static void main(String[] args) {  
    A a = new A();  
    A b = new B();  
    A c = new C(3);  
    System.out.println(a.toString());  
    System.out.println(b.toString());  
    System.out.println(c.toString());  
}
```

```
public class A {  
    private int x;  
    A(){  
        this.x = -1;  
        System.out.println("A()");  
    }  
    public int getX() {return x;}  
    public void setX(int x) {this.x = x;}  
    public String toString(){  
        return ("x");  
    }  
}
```

```
public class B extends A {  
    private int y;  
    public int getY() { return y; }  
    public void setY(int y) {this.y = y;}  
    B(){  
        this.y=1;  
        System.out.println("B()");  
    }  
}
```

```
public String toString(){return ("super.toString()+"  
y+"");}
```

```
public class C extends B {  
    private int z;  
    C(int z){  
        this.z=z;  
        System.out.println("C()");  
    }  
    public int getZ() { return z; }  
    public void setZ(int z) { this.z = z;}  
    public String toString(){return ("["+super.toString()+"]");}
```



ISCAE 2017/2018

CC JAVA II

S3 ID, IG et RT

ISCAE 2017/2018

CC JAVA II

S3 ID, IG et RT

Question 1 (3 points) : la classe `ClePerso`

Soit n votre matricule d'étudiant sans le préfixe I, par exemple, si votre matricule est 02480 plus n = 02480

- Donner les deux nombres : `int x = n % 50` et `int y = n/2`
- Compléter la classe suivante en mettant les deux valeurs obtenues ci-dessus

```

public class ClePerso {
    public static int x = ....;
    public static int y = ....;
}

```

Question 2 (5 points) Corrigez les erreurs de compilation dans le programme suivant. Les lignes des lignes et en argumentant

```

package mr.iscae.ig;

public abstract class A {
    protected int x;
    public A(){
        this(-1,0);
    }
    public A(int x){
        this.x = x;
    }
    public abstract void m();
    public String toString() {
        return ("x="+x);
    }
}

class B extends A {
    public String toString() {
        return ("x="+x);
    }
}

```

```

package mr.iscae.ig;

public class C extends A {
    int y;
    public void m(){
        System.out.println(this.y);
    }
}

class D extends C {
    public D(int y){
        super(2);
        this.y = y;
    }
}

```

```

package mr.iscae.rt;

public abstract class E {
    extends {
        public static void main(String
        [] args) {
            A[] t = new A[3];
            t[0] = new A(0);
            t[1] = new A(1);
            t[2] = new A(2);
        }
    }
}

class F extends E {
    public static void main(String
    [] args) {
        A[] t = new A[3];
        t[0] = new A(0);
        t[1] = new A(1);
        t[2] = new A(2);
    }
}

```

```

public class Exo3 {
    public static void main(String [] args){
        A[] tab = new A[3];
        tab[0] = new A(1);
        tab[1] = new B(-1);
        tab[2] = new C(2);
        for(int i=0; i<tab.length; i++){
            System.out.println(tab[i]);
        }
    }
}

```

Question 4 (7 points) Ecrire un programme java qui fait ce qui suit :

- Declare une classe `Point` avec deux attributs `x` et `y` de type double (coordonnées d'un point)
- Ecrire, les setters, getters, constructeur avec deux arguments qui initialise les deux attributs, constructeur sans argument qui crée un point de coordonnées (`ClePerso.x`, `ClePerso.y`) et un constructeur par copie
- Puis, on considère les classes : Quadrilatere, Trapèze, Parallélogramme, Rectangle, Losange
 - Ecrire la classe `Quadrilatere` qui a quatre points comme attributs et faire les setters, getters, et constructeur par copie
 - Ecrire les autres classes en respectant au maximum l'hérarchie entre elles (Trapèze est un quadrilatere, Rectangle est un parallélogramme, etc.) (Ces classes ne contiennent pas d'attributs supplémentaires)
 - On considère le code suivant :


```

classintere {
    tab = Quadrilatere[4];
    for(int i=0; i<4; i++){
        switch(i) {
            case 0: tab[i] = new Trapèze(); break;
            case 1: tab[i] = new Parallélogramme(); break;
            case 2: tab[i] = new Rectangle(); break;
            case 3: tab[i] = new Losange(); break;
        }
        System.out.println(" "+tab[i]);
    }
}

```

 Apposer les modifications nécessaires pour que ce code marche et affiche T P R L

Question 3 (5 points) en exécutant le code ci-dessous, donnez le résultat qui s'affiche sur l'écran

```

class A {
    private int x;
    int y;
    A(){
        x = 1;
        y = -1;
        System.out.println("A");
    }
    public String toString() {
        return "A";
    }
}

class B extends A {
    private int z;
    B(){
        z = 2;
    }
}

```

Question 1 (6 points) : Choisissez la bonne réponse pour chacune des questions suivantes (Attention une mauvaise réponse coûte -2 points)

- | | | |
|--|---|--|
| <p>1) Une classe qui contient une méthode abstraite</p> <p>a) Ne peut pas avoir un constructeur</p> <p>b) On ne peut pas l'instancier</p> <p>c) On doit la déclarer en étant une classe abstraite</p> <p>d) a) et b)</p> <p>e) a) et c)</p> <p>f) b) et c)</p> | <p>2) Une société de transport veut stocker des informations sur ses véhicules : nombre de portes, climatisé ou non, vitesse maximum. Quelle est la bonne conception ?</p> <p>a) Déclarer quatre classes indépendantes : Vehicule, Porte, Climatise, Vitesse</p> <p>b) Utiliser une classe Vehicule avec des classes filles : Porte, Climatise, Vitesse</p> <p>c) Utiliser une classe Vehicule avec les attributs : nbPortes, estClimatise, vitesse</p> | <p>3) Que s'affiche t-il sur l'écran en exécutant les instructions :
 Mere p = new Fille(); p.m1();
 En considérant les classes :
 public class Mere{
 public void m1(){ System.out.println("A"
); m2(); }
 public void m2(){System.out.println("B"); }

 public class Fille extends Mere{
 public void m1(){
 super.m1();System.out.println("C"); }
 public void
 m2(){super.m2();System.out.println("D"
); }
 }
 a) ABDC b) AB c) ABCD d) ABC</p> |
|--|---|--|

Question2 (6 points) : Corrigez les erreurs dans le code suivant

```

1 public class TestEre {
2     public static void main(String args[]){
3         J j = new J();
4         I i = new I();
5         A a = new A();
6     }
7 }
8
9 abstract class A {
10     A() {
11         System.out.println("A");
12     }
13 }
14
15 interface I {
16     I() {
17
18         boolean p();
19         boolean g();
20         protected int e();
21     }
22 }
23 interface J {
24 }
25
26 class Test1 extends I, J {
27 }
28
29 class Test2 implements I {
30 }
31
32 class Test3 extends A {
33 }

```

Question 3 (8 points) : En utilisant l'API Swing, écrire un programme JAVA qui :

1. Présente à l'utilisateur une interface graphique permettant de donner 4 notes pour quatre matières (à saisir dans des champs de saisie) et pour chaque note, on choisit le coefficient (une liste de 3 choix (2, 1, 3) pour chaque matière. C'est-à-dire l'utilisateur n'écrit pas le coefficient pour une matière, mais il choisit entre 2, 1 ou 3.
2. L'interface doit avoir un bouton permettant d'afficher la moyenne dès qu'on clique. L'affichage de la moyenne doit se faire dans un champ de saisie non éditable. La moyenne calculée est inférieure à 12 on affiche à droite de la fenêtre le message Admis(e).



Question 1 (10 points) : des erreurs de compilation apparaissent dans le code ci-dessous. Proposez une correction du code sans supprimer des instructions.

```
public abstract class A {
    public abstract void m();
    public int v(int a){ return v*2;}
}
public class B implements A {
    int b;
    B () {
        super(1);
    }
    B ( int c) {
        this.b =b;}
}
```

```
public interface C {
    private int met(int a) ;
}
public class D extends C, implements A
{
    int c;
    public D(){ super("b"); }
}
public class Test{
    public static void main(String [] args){
        A a =new A();
        C c = new B();
    }
}
```

Question 2 (10 pts) : En utilisant l'API Swing, écrire un programme JAVA qui :

1. Présente à l'utilisateur une interface graphique permettant de donner 3 notes pour trois matières (à saisir dans un champ de saisie) et pour chaque note choisir le coefficient (une liste de 3 choix (1, 3, 4) pour chaque matière). C'est-à-dire l'utilisateur n'écrit pas le coefficient pour une matière, mais il choisit entre 1, 3 ou 4.
2. L'interface doit avoir un bouton permettant d'afficher la moyenne dès qu'on clique dessus. L'affichage de la moyenne doit se faire dans un champ de saisie non éditable.
3. Si la moyenne calculée est inférieure à 10 on affiche en bas de la fenêtre le message **Ajourné(e)** sinon le message **Admis(e)**.



Bonne chance !